

CST2550 Group Coursework Report
Cinema Film Booking and Membership System
Adidas Group

Introduction

The Cinema Film Booking and Membership System is a web application built using C# ASP.NET Core that allows customers to browse films, register an account, log in, purchase tickets online and delete their account. The system supports three membership tiers (Standard, Premium, and VIP) each offering different discount rates at checkout. Customers can view available films with details like showtime, genre, rating and price, then purchase a ticket by entering their payment card details and billing address. After purchasing, customers can log in and view all their tickets.

The application uses custom-built data structures on the backend to store and manage data efficiently in memory while it is running. Film data is stored in a custom hash table for fast lookups by ID, while customer accounts and tickets are stored in custom linked lists. Data is persisted using a Microsoft SQL Server LocalDB database, with all reads and writes handled through ADO.NET. This means customer accounts and purchased tickets are saved between sessions, so users can come back later and still see their tickets. The system validates all user input including ensuring card numbers are 16 digits, CVV codes are 3 digits, and expiry month and year are entered as numbers only.

This report covers the design decisions behind the chosen data structures with time complexity analysis, the testing approach used to verify correctness, and a reflection on what was achieved and what could be improved in future.

Design

Data structure selection

Hash table for films

We have chosen a hash table with separate chaining for storing films. Films are looked up by their ID when a customer wants to buy a ticket, and a hash table provides $O(1)$ time complexity for both insertion and lookup (Cormen et al, 2009). The cinema has a fixed set of films loaded at start-up, making a hash table ideal since it handles this kind of dataset efficiently.

The hash function takes the film ID, computes its hash code, and maps it to a bucket index using the modulo operator. When two keys hash to the same index (a collision), the new entry is added to the front of a linked chain at that bucket. This is called separate chaining and we chose this because it is straightforward to implement and handles collisions without needing to resize the table for empty slots (Eissa, 2025a).

Linked list for customers and tickets

A linked list was chosen for storing customer accounts and tickets. Unlike films, customers and tickets do not need constant lookup by a numeric key. The system searches customers by email during login and filters tickets by customer ID when viewing purchased tickets. A linked list supports $O(1)$ insertion and $O(n)$ traversal, which works for the expected data size of a cinema system.

The linked list is also dynamic, meaning it grows as new customers register and new tickets are purchased without needing to allocate a fixed amount of memory beforehand (Eissa, 2025b).

Database Design

The system uses a SQL Server LocalDB database called CinemaDB with three tables: Films, Customers, and Tickets. The Films table holds the catalogue of available films with their details including a poster image URL. The Customers table stores account information with an email and password for login. The Tickets table holds purchased tickets with a foreign key reference to both the customer who bought it and the film they bought a ticket for. This was designed to follow basic normalisation principles (Eissa, 2025d).

Algorithm Design (Pseudo Code)

Hash Table Insert

```
# insert(key, value)

FUNCTION insert(key, value)
    index = HashFunction(key) MOD tableSize
    current = buckets[index]

    WHILE current is not NULL
        IF current.key EQUALS key THEN
            current.value = value
            RETURN
        END IF
        current = current.next
    END WHILE

    newNode = CREATE new HashNode(key, value)
    newNode.next = buckets[index]
    buckets[index] = newNode
    count = count + 1
END FUNCTION
```

Hash Table Search

```
# search(key)

FUNCTION search(key)
    index = HashFunction(key) MOD tableSize
    current = buckets[index]

    WHILE current is not NULL
        IF current.key EQUALS key THEN
            RETURN current.value
        END IF
        current = current.next
    END WHILE

    RETURN null
END FUNCTION
```

Hash Table Delete

```
# delete(key)

FUNCTION delete(key)
    index = HashFunction(key) MOD tableSize
    current = buckets[index]
    previous = NULL

    WHILE current is not NULL
        IF current.key EQUALS key THEN
            IF previous is NULL THEN
                buckets[index] = current.next
            ELSE
                previous.next = current.next
            END IF
            count = count - 1
            RETURN true
        END IF
        previous = current
        current = current.next
    END WHILE

    RETURN false
END FUNCTION
```

Linked List Insert at Head

```
# InsertAtHead(data)

FUNCTION InsertAtHead(data)
    newNode = CREATE new ListNode(data)
    newNode.next = head
    head = newNode
    count = count + 1
END FUNCTION
```

Linked List Insert at Tail

```
# InsertAtTail(data)

FUNCTION InsertAtTail(data)
    newNode = CREATE new ListNode(data)

    IF head is NULL THEN
        head = newNode
    ELSE
        current = head
        WHILE current.next is not NULL
            current = current.next
        END WHILE
        current.next = newNode
    END IF

    count = count + 1
END FUNCTION
```

Linked List Delete

```
# delete(condition)

FUNCTION delete(condition)
  IF head is NULL THEN
    RETURN false
  END IF

  IF condition(head.data) is true THEN
    head = head.next
    count = count - 1
    RETURN true
  END IF

  current = head
  WHILE current.next is not NULL
    IF condition(current.next.data) is true THEN
      current.next = current.next.next
      count = count - 1
      RETURN true
    END IF
    current = current.next
  END WHILE

  RETURN false
END FUNCTION
```

Time Complexity Analysis

```
# time complexity
```

Operation	Average Case	Worst Case
Hash Table Insert	$O(1)$	$O(n)$
Hash Table Search	$O(1)$	$O(n)$
Hash Table Delete	$O(1)$	$O(n)$
Linked List InsertHead	$O(1)$	$O(1)$
Linked List InsertTail	$O(n)$	$O(n)$
Linked List Search	$O(n)$	$O(n)$
Linked List Delete	$O(n)$	$O(n)$

The hash table achieves $O(1)$ average case because with a good hash function and appropriately sized table, each bucket contains very few elements. The worst case of $O(n)$ occurs when all keys hash to the same bucket, forming a single long chain. The linked list operations are $O(n)$ for search and delete because the list must be traversed from the head until the target element is found.

Testing

Testing Approach

Unit testing was carried out using Microsoft's MSTest framework. Each core component was tested independently to verify it works correctly before being integrated into the web application. The tests cover the model classes (Film, Customer, Ticket), the custom data structures (hash table and linked list), and business logic (membership discount calculations and final price computation).

All tests follow the Arrange-Act-Assert pattern where test data is set up, the operation is performed, and the result is compared against the expected output (Eissa, 2025c).

Test Cases

Test ID	Description	Input	Expected output	Result
T01	Film constructor sets all properties	Film(1, "Inception", "Sci-Fi", 148, "12A", "18:30", 12.99, 50)	All properties match	Pass
T02	Default Film constructor	new Film()	Empty strings, 50 seats	Pass
T03	Film Equals with same ID	Two films with ID 1	Returns true	Pass
T04	Film Equals with different ID	Films with ID 1 and 2	Returns false	Pass
T05	Hash table insert and count	Insert one item	Count = 1	Pass
T06	Hash table search existing key	Insert then search key 1	Returns correct value	Pass
T07	Hash table search missing key	Search for key 99	Returns null	Pass
T08	Hash table delete	Delete existing key	Returns true, Count decreases	Pass
T09	Hash table update same key	Insert key 1 twice	Value updated, count stays 1	Pass
T10	Linked list insert at head	Insert two items	Count = 2	Pass
T11	Linked list search existing	Search for value 20	Returns 20	Pass

T12	Linked list delete existing	Delete value 10	Returns true, Count decreases	Pass
T13	Linked list to array	Insert 3 items	Array length = 3, correct order	Pass
T14	Customer discount Standard	MembershipType = "Standard"	Discount = 0%	Pass
T15	Customer discount Premium	MembershipType = "Premium"	Discount = 10%	Pass
T16	Customer discount VIP	MembershipType = "VIP"	Discount = 20%	Pass
T17	VIP price calculation	Film £10.00, VIP customer	Final price = £8.00	Pass
T18	Ticket constructor	Ticket(1, 10, 5, "Inception", 3, 12.99)	All properties match	Pass
T19	Ticket default date	new Ticket()	Today's date	Pass

Conclusion

Summary

The Cinema Film Booking and Membership System was successfully built as a C# ASP.NET Core web application using Razor Pages. Users can register an account, log in, browse and search films, purchase tickets with validated payment details, and view their purchased tickets. Custom data structures were implemented from scratch on the backend including a hash table with separate chaining for film storage and a linked list for customer and ticket management. The web frontend uses HTML and CSS with to display data from the C# backend. Input validation ensures card numbers are 16 digits, CVV codes are 3 digits, and expiry fields only accept numbers.

Limitations

Payment details are stored in plain text which would not be acceptable in a real production system. Card data should be encrypted or handled through a secure payment provider. The system does not check if a seat is already booked by another customer, so duplicate seat bookings are possible. There is no password hashing so customer passwords are stored as plain text. The hash table has a fixed size that does not automatically resize based on load factor, which could cause performance issues with a very large film catalogue.

Future Improvements

Payment processing should use a secure third-party provider rather than storing card details directly. Adding password hashing would improve account security.

A binary search tree could replace the hash table to allow range queries like finding all films under a certain price. Implementing a proper seat map that shows which seats are already taken would improve the booking experience.

References

Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2009) Introduction to Algorithms. 3rd edn. Cambridge, MA: MIT Press.

Eissa, A. (2025a) 'CST2550 Week 4: Advanced Data Structures - Graphs and Hash Tables', Middlesex University London.

Eissa, A. (2025b) 'CST2550 Week 3: Data Structures - Arrays and Lists', Middlesex University London.

Eissa, A. (2025c) 'CST2550 Week 7: Debugging and Testing Software', Middlesex University London.

Eissa, A. (2025d) 'CST2550 Week 5: Introduction to Database and SQL', Middlesex University London.

Eissa, A. (2025d) 'CST2550 Week 6: Implementing Database in C# Applications', Middlesex University London.

Microsoft (2024) ADO.NET Overview. Available at:
<https://learn.microsoft.com/enus/dotnet/framework/data/adonet/>

Microsoft (2024) ASP.NET Core Razor Pages. Available at:
<https://learn.microsoft.com/enus/aspnet/core/razor-pages/>

Microsoft (2024) MSTest framework. Available at:
<https://learn.microsoft.com/enus/dotnet/core/testing/unit-testing-with-mstest>